

Beginner Explanation of the ICKAN Test on the Neo-Hookean RVE Database

Sebastian ICKAN Tests

June 29, 2026

Contents

1	Purpose of this note	1
2	Very short summary	1
3	Files involved	2
4	The physical problem behind the data	2
4.1	What is the RVE?	2
4.2	Why do we need homogenization?	3
5	Basic mechanics vocabulary	3
5.1	Reference and current configuration	3
5.2	Deformation gradient	3
5.3	Right Cauchy–Green tensor	3
5.4	Green–Lagrange strain	4
6	From strain to boundary displacement	4
6.1	Why the solver needs the deformation gradient $\mathbf{F}$	4
6.2	Why choose $\mathbf{F}$ equal to the square root of $\mathbf{C}$?	4
6.3	Boundary displacement	5
7	Stress measure	5
7.1	What stress is stored?	5
8	Principal stretches	5
8.1	What is a principal direction?	5
8.2	How do principal stretches come from $\mathbf{C}$?	5
9	Invariants	6
9.1	Beginner definition	6
9.2	Why invariants matter for machine learning	6
9.3	What invariants are used in the current ICKAN code?	6
10	Isochoric and volumetric split	6
10.1	What does isochoric mean?	6
10.2	Why multiply by J to the power minus one third?	7
10.3	Important plane-strain subtlety	7

11 Hyperelasticity	7
11.1 What is a hyperelastic material?	7
11.2 Why learn energy instead of stress directly?	7
11.3 What does the current ICKAN model learn?	8
12 What is a KAN?	8
12.1 Standard neural network intuition	8
12.2 KAN intuition	8
12.3 What does <code>grid_size</code> mean?	9
12.4 What does spline degree k mean?	9
13 What is ICKAN in this repository?	9
13.1 Input size	9
13.2 Network width	9
13.3 Zero-energy normalization	9
13.4 Zero-stress normalization	10
14 How the training data is built	10
14.1 Loading trajectories	10
14.2 Downsampling	10
14.3 Normalization	10
14.4 Energy reconstruction	11
14.5 Important path-dependence warning	11
15 Training objective	11
15.1 The available training modes	11
15.2 Why is this still energy-based?	11
16 Convexity	12
16.1 Convexity in one dimension	12
16.2 Convexity in several dimensions	12
16.3 What would convex energy imply?	12
16.4 Is convexity always required?	12
17 Monotonicity	12
17.1 One-dimensional monotonicity	12
17.2 Tangent interpretation	13
17.3 Multidimensional monotonicity	13
18 Polyconvexity	13
18.1 Why polyconvexity exists	13
18.2 Beginner definition	13
18.3 Why people care about polyconvexity	13
18.4 Does the current ICKAN guarantee polyconvexity?	14
19 How to read the current ICKAN code	14
19.1 <code>ICKAN_surrogate.py</code>	14
19.2 <code>run_ICKAN_surrogate.py</code>	15

20 Important caveats in the current scripts	15
20.1 Caveat 1: external dependency path	15
20.2 Caveat 2: output directory	15
20.3 Caveat 3: computed strain versus applied strain	15
20.4 Caveat 4: normalization and physical features	16
20.5 Caveat 5: no train/test split	16
20.6 Caveat 6: not certified convex or polyconvex	16
21 What would a good ICKAN test report include?	17
22 Recommended next steps	17
22.1 Step 1: make the code portable	17
22.2 Step 2: create output directories	17
22.3 Step 3: decide the input strain	17
22.4 Step 4: save scalars	18
22.5 Step 5: add a real test	18
23 Mental model for Sebastian	18
24 Glossary	19
25 Final interpretation	19

1 Purpose of this note

This note explains, from the beginning, what the ICKAN test is trying to do with the RVE database generated in this repository. It assumes that the reader is new to:

- finite strain mechanics,
- hyperelasticity,
- homogenization,
- principal stretches,
- invariants,
- convexity, monotonicity, and polyconvexity,
- KAN/ICKAN-type neural networks.

The goal is not only to explain the code, but also to explain the mechanics ideas behind the code. The main message is:

The ICKAN scripts try to learn an effective hyperelastic constitutive law of the perforated RVE. The input is a macroscopic strain state. The output is the macroscopic homogenized stress. Instead of predicting stress directly, the ICKAN predicts a scalar energy, and stress is obtained by differentiating that energy.

2 Very short summary

The Stage 1 FOM database contains pairs of the form

$$\text{strain} \longmapsto \text{stress}.$$

More specifically, every sample looks like

$$\mathbf{E} = \begin{bmatrix} E_{xx} \\ E_{yy} \\ G_{xy} \end{bmatrix} \longmapsto \mathbf{S} = \begin{bmatrix} S_{xx} \\ S_{yy} \\ S_{xy} \end{bmatrix}.$$

Here:

- $\mathbf{E}$ is the Green–Lagrange strain,
- G_{xy} is the engineering shear strain, so $G_{xy} = 2E_{xy}$,
- $\mathbf{S}$ is the second Piola–Kirchhoff stress,
- both are homogenized, meaning they are average macroscopic quantities extracted from the detailed RVE finite element solution.

The ICKAN code tries to learn a scalar function

$$W = W(\mathbf{E}),$$

where W is an effective strain energy density. Then stress is computed as

$$\mathbf{S} = \frac{\partial W}{\partial \mathbf{E}}.$$

This is the key idea. The neural network does not directly output three stress components. It outputs one scalar energy. The stress is obtained from the gradient of this energy with respect to strain.

3 Files involved

The relevant files are:

- `stage_1_training.set_fom/trajectory_i/trajectory_i_strain.npy`
- `stage_1_training.set_fom/trajectory_i/trajectory_i_stress.npy`
- `stage_1_training.set_fom/trajectory_i/trajectory_i_applied_strain.npy`
- `ICKAN_surrogate.py`
- `run_ICKAN_surrogate.py`

The ICKAN scripts currently load `trajectory_i_strain.npy` and `trajectory_i_stress.npy`. They do not use the displacement field `trajectory_i_U.npy`.

4 The physical problem behind the data

4.1 What is the RVE?

RVE means representative volume element. In this repository, the RVE is a small two-dimensional sample of material:

- the outer domain is rectangular/square,
- there is a circular hole in the center,
- the mesh is fixed,
- the material is hyperelastic and plane strain,
- different macroscopic strain states are imposed on the boundary.

The geometry is not being varied in the ICKAN test. The hole size, mesh, and material parameters are fixed. What changes from sample to sample is the imposed macroscopic strain.

4.2 Why do we need homogenization?

The RVE has local stress and strain fields that vary from point to point. Near the hole, stress concentrations appear. Farther from the hole, the response is different.

However, for a macroscopic material model, we do not want to store every local finite element value. We want an average effective law:

$$\text{macroscopic strain} \mapsto \text{macroscopic stress}.$$

Homogenization turns the detailed finite element result into average quantities. The resulting effective material is not simply the original Neo-Hookean material because the hole changes the macroscopic behavior. The ICKAN is trying to learn this effective behavior.

5 Basic mechanics vocabulary

5.1 Reference and current configuration

In finite deformation mechanics, we distinguish between:

- the reference configuration: the undeformed geometry,
- the current configuration: the deformed geometry.

A material point starts at reference position

$$\mathbf{X} = \begin{bmatrix} X \\ Y \end{bmatrix}.$$

After deformation, it moves to

$$\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}.$$

The displacement is

$$\mathbf{u} = \mathbf{x} - \mathbf{X}.$$

5.2 Deformation gradient

The deformation gradient is

$$\mathbf{F} = \frac{\partial \mathbf{x}}{\partial \mathbf{X}}.$$

It tells us how an infinitesimal material vector changes during deformation.

For small deformation, one often talks directly about displacement gradients. For large deformation, $\mathbf{F}$ is more fundamental.

5.3 Right Cauchy–Green tensor

From $\mathbf{F}$, we define

$$\mathbf{C} = \mathbf{F}^T \mathbf{F}.$$

This is called the right Cauchy–Green tensor. It measures stretching while removing rigid-body rotation.

Why does $\mathbf{C}$ remove rotations? Suppose the body only rotates and does not stretch. Then

$$\mathbf{F} = \mathbf{R},$$

where $\mathbf{R}$ is a rotation matrix. A rotation satisfies

$$\mathbf{R}^T \mathbf{R} = \mathbf{I}.$$

Therefore

$$\mathbf{C} = \mathbf{F}^T \mathbf{F} = \mathbf{R}^T \mathbf{R} = \mathbf{I}.$$

So $\mathbf{C}$ says “no stretch”, which is correct.

5.4 Green–Lagrange strain

The Green–Lagrange strain is

$$\mathbf{E} = \frac{1}{2}(\mathbf{C} - \mathbf{I}).$$

Equivalently,

$$\mathbf{C} = 2\mathbf{E} + \mathbf{I}.$$

This is the strain measure used in your data. In two dimensions:

$$\mathbf{E} = \begin{bmatrix} E_{xx} & E_{xy} \\ E_{xy} & E_{yy} \end{bmatrix}.$$

The code stores strain in Voigt form:

$$\mathbf{E}_{\text{voigt}} = \begin{bmatrix} E_{xx} \\ E_{yy} \\ G_{xy} \end{bmatrix}, \quad G_{xy} = 2E_{xy}.$$

Therefore, when the code reconstructs the tensor, it does:

$$E_{xy} = \frac{1}{2}G_{xy}.$$

6 From strain to boundary displacement

6.1 Why the solver needs the deformation gradient $\mathbf{F}$

The finite element solver imposes displacements on the boundary. But the dataset is defined in terms of strain $\mathbf{E}$. So the solver must convert $\mathbf{E}$ into a deformation gradient $\mathbf{F}$.

The code uses:

$$\mathbf{C} = 2\mathbf{E} + \mathbf{I},$$

then

$$\mathbf{F} = \mathbf{C}^{1/2}.$$

Here $\mathbf{C}^{1/2}$ means the symmetric positive definite square root of $\mathbf{C}$.

6.2 Why choose $\mathbf{F}$ equal to the square root of $\mathbf{C}$?

Many different deformation gradients can have the same $\mathbf{C}$. For example:

$$\mathbf{F} = \mathbf{R}\mathbf{C}^{1/2}$$

has the same $\mathbf{C}$ for any rotation $\mathbf{R}$, because

$$\mathbf{F}^T \mathbf{F} = (\mathbf{R}\mathbf{C}^{1/2})^T (\mathbf{R}\mathbf{C}^{1/2}) = \mathbf{C}^{1/2} \mathbf{R}^T \mathbf{R} \mathbf{C}^{1/2} = \mathbf{C}.$$

The code chooses the rotation-free representative:

$$\mathbf{F} = \mathbf{C}^{1/2}.$$

This is a good choice for machine learning because rigid rotations should not affect the material law. If rotations were included arbitrarily, the same strain could correspond to many different boundary displacements, making the learning problem messier.

6.3 Boundary displacement

The imposed affine displacement is

$$\mathbf{u}_{\text{aff}} = (\mathbf{F} - \mathbf{I})(\mathbf{X} - \mathbf{X}_c),$$

where $\mathbf{X}_c$ is the center of the RVE domain. This means the boundary is moved in a way consistent with the desired macroscopic strain.

7 Stress measure

7.1 What stress is stored?

The stored stress is the second Piola–Kirchhoff stress:

$$\mathbf{S}.$$

This stress is naturally paired with the Green–Lagrange strain in energy calculations:

$$dW = \mathbf{S} : d\mathbf{E}.$$

The double dot product means:

$$\mathbf{S} : d\mathbf{E} = S_{xx} dE_{xx} + S_{yy} dE_{yy} + 2S_{xy} dE_{xy}.$$

Since the code stores $G_{xy} = 2E_{xy}$, the work expression in Voigt form becomes

$$dW = S_{xx} dE_{xx} + S_{yy} dE_{yy} + S_{xy} dG_{xy}.$$

This explains why the code can use the simple dot product:

$$\Delta W \approx \mathbf{S}_{\text{voigt}} \cdot \Delta \mathbf{E}_{\text{voigt}}.$$

8 Principal stretches

8.1 What is a principal direction?

Imagine stretching a square rubber sheet. In some directions the material may be stretched more, and in other directions less. The special directions where the stretch happens without shear are called principal directions.

The stretch factors in those directions are called principal stretches:

$$\lambda_1, \lambda_2, \lambda_3.$$

If $\lambda_i > 1$, the material is stretched in that direction. If $\lambda_i < 1$, it is compressed in that direction.

8.2 How do principal stretches come from $\mathbf{C}$?

The eigenvalues of $\mathbf{C}$ are the squares of the principal stretches:

$$\text{eigenvalues of } \mathbf{C} = \lambda_i^2.$$

Therefore:

$$\lambda_i = \sqrt{\text{eigenvalue}_i(\mathbf{C})}.$$

The ICKAN code does this:

```
square_eigenvalues = torch.linalg.eigvalsh(C)
eigenvalues = torch.sqrt(square_eigenvalues)
```

Despite the variable name `eigenvalues`, this quantity is actually the principal stretches λ_i .

9 Invariants

9.1 Beginner definition

An invariant is a quantity that does not change when we rotate the coordinate system.

For example, suppose you describe a deformation in the x - y axes, then rotate your axes and describe the same deformation again. The tensor components change, but the physical deformation is the same. A good material model should not change its prediction just because we changed the coordinate system.

Quantities such as:

$$\lambda_1, \lambda_2, \lambda_3$$

or combinations such as:

$$I_1 = \text{tr}(\mathbf{C}), \quad I_2 = \frac{1}{2} [(\text{tr} \mathbf{C})^2 - \text{tr}(\mathbf{C}^2)], \quad I_3 = \det(\mathbf{C})$$

are invariants.

9.2 Why invariants matter for machine learning

If a neural network learns directly from tensor components, it may learn a coordinate-dependent representation. That can be inefficient and physically fragile.

If the network learns from invariants or principal stretches, it is encouraged to learn a material response that depends on physical deformation, not on the chosen coordinate axes.

9.3 What invariants are used in the current ICKAN code?

The code does not use I_1, I_2, I_3 explicitly. Instead, it uses stretch-like features:

$$\bar{\lambda}_1 = \lambda_1 J^{-1/3}, \quad \bar{\lambda}_2 = \lambda_2 J^{-1/3},$$

and a volumetric feature:

$$(J - 1)^2.$$

Here

$$J = \sqrt{\det(\mathbf{C})}.$$

In three-dimensional finite strain mechanics, $J = \det(\mathbf{F})$ is the volume change. If $J = 1$, the deformation preserves volume. If $J > 1$, the material locally expands. If $J < 1$, it locally compresses.

10 Isochoric and volumetric split

10.1 What does isochoric mean?

Isochoric means volume-preserving. It describes the part of the deformation that changes shape but not volume.

Volumetric means related to volume change.

Many hyperelastic models split the energy into two parts:

$$W = W_{\text{iso}} + W_{\text{vol}}.$$

The isochoric part controls distortional/shear-like deformation. The volumetric part controls compression or expansion.

10.2 Why multiply by J to the power minus one third?

In three dimensions, the product of the three principal stretches is:

$$J = \lambda_1 \lambda_2 \lambda_3.$$

The modified stretches

$$\bar{\lambda}_i = J^{-1/3} \lambda_i$$

have product:

$$\bar{\lambda}_1 \bar{\lambda}_2 \bar{\lambda}_3 = J^{-1} \lambda_1 \lambda_2 \lambda_3 = 1.$$

So the modified stretches remove the volume change and keep only the shape change.

10.3 Important plane-strain subtlety

Your problem is two-dimensional plane strain, but the constitutive law is still a finite-strain solid law. In plane strain, one can think of an out-of-plane stretch $\lambda_3 = 1$. Then:

$$J = \lambda_1 \lambda_2 \lambda_3 = \lambda_1 \lambda_2.$$

The code uses a two-dimensional $\mathbf{C}$ and computes:

$$J = \sqrt{\det(\mathbf{C}_{2D})}.$$

This is compatible with the idea $\lambda_3 = 1$, but it is worth remembering that the ICKAN input only includes two in-plane modified stretches plus one volumetric term.

11 Hyperelasticity

11.1 What is a hyperelastic material?

A hyperelastic material is a material where stress comes from an energy function. For Green–Lagrange strain and second Piola–Kirchhoff stress:

$$\mathbf{S} = \frac{\partial W}{\partial \mathbf{E}}.$$

This is similar to a spring. For a linear spring:

$$W(u) = \frac{1}{2}ku^2, \quad f = \frac{dW}{du} = ku.$$

Energy gives force by differentiation.

For a hyperelastic solid:

$$W(\mathbf{E}) \Rightarrow \mathbf{S}(\mathbf{E}).$$

11.2 Why learn energy instead of stress directly?

There are three main reasons.

Reason 1: mechanical consistency. If stress is the gradient of an energy, the model is automatically conservative. That means the stress comes from a potential.

Reason 2: symmetric tangent stiffness. The material tangent is the derivative of stress with respect to strain:

$$\mathbb{C}_{\text{tan}} = \frac{\partial \mathbf{S}}{\partial \mathbf{E}} = \frac{\partial^2 W}{\partial \mathbf{E}^2}.$$

Second derivatives of a smooth scalar potential are symmetric. This is a useful physical and numerical property.

Reason 3: better extrapolation structure. A scalar energy model can encode physical structure more naturally than three independent stress outputs.

11.3 What does the current ICKAN model learn?

The model learns:

$$W_{\theta}(\mathbf{E}),$$

where θ denotes the trainable KAN parameters. Then it computes:

$$\mathbf{S}_{\theta}(\mathbf{E}) = \frac{\partial W_{\theta}}{\partial \mathbf{E}}$$

using PyTorch automatic differentiation.

12 What is a KAN?

12.1 Standard neural network intuition

A standard multilayer perceptron uses linear combinations of inputs followed by activation functions. Schematically:

$$\mathbf{z}_{\ell+1} = \sigma(\mathbf{W}_{\ell}\mathbf{z}_{\ell} + \mathbf{b}_{\ell}).$$

The trainable objects are mostly the matrix weights $\mathbf{W}_{\ell}$ and biases $\mathbf{b}_{\ell}$.

12.2 KAN intuition

KAN stands for Kolmogorov–Arnold Network. In a KAN, the trainable functions are often placed on edges of the network. Instead of just multiplying by a scalar weight, an edge can apply a learnable one-dimensional function.

Very roughly, a KAN layer can be imagined as:

$$z_j^{\ell+1} = \sum_i \phi_{ij}^\ell(z_i^\ell),$$

where each ϕ_{ij}^ℓ is a learnable scalar function.

In many KAN implementations, these scalar functions are represented by splines. That is why the code talks about:

- grid size,
- spline degree k ,
- grid update from samples.

12.3 What does grid_size mean?

The spline functions live on a grid. A larger grid gives more local flexibility, but can overfit and increases cost.

In the current code:

```
grid_size = 4.
```

This is quite small, meaning the network is intentionally simple.

12.4 What does spline degree k mean?

The code uses:

```
k = 3.
```

This means cubic splines. Cubic splines are smooth piecewise-polynomial functions. Smoothness is important because the model differentiates energy to obtain stress.

13 What is ICKAN in this repository?

The file `ICKAN_surrogate.py` defines a class:

```
ICKAN_W_Surrogate.
```

The “W” in the name is important. This is an energy surrogate.

13.1 Input size

The code defines:

```
input_size = 2 * order_stretches + 1.
```

With the current setting:

```
order_stretches = 1,
```

so:

```
input_size = 3.
```

The three inputs are:

$$\bar{\lambda}_1, \quad \bar{\lambda}_2, \quad (J-1)^2.$$

13.2 Network width

The current code uses:

$$\text{W_width} = [3, 2, 1, 1].$$

This means the KAN maps:

$$3 \longrightarrow 2 \longrightarrow 1 \longrightarrow 1.$$

The final output is a scalar:

$$W_\theta.$$

13.3 Zero-energy normalization

The code subtracts the network output at zero strain:

$$W_\theta(\mathbf{E}) - W_\theta(\mathbf{0}).$$

This enforces:

$$W_\theta(\mathbf{0}) = 0.$$

That is physically reasonable because the undeformed state should have zero stored energy, up to an arbitrary additive constant.

13.4 Zero-stress normalization

The code also subtracts the stress at zero strain:

$$\mathbf{S}_\theta(\mathbf{E}) - \mathbf{S}_\theta(\mathbf{0}).$$

This enforces:

$$\mathbf{S}_\theta(\mathbf{0}) = \mathbf{0}.$$

That is also physically reasonable for an initially stress-free RVE.

14 How the training data is built

14.1 Loading trajectories

The script `run_IKAN_surrogate.py` loads ten trajectories:

$$i = 1, \dots, 10.$$

For each trajectory it loads:

$$\text{trajectory_i_strain.npy}, \quad \text{trajectory_i_stress.npy}.$$

These files contain arrays of shape:

$$N_i \times 3.$$

The number of steps N_i differs by trajectory.

14.2 Downsampling

The script sets:

$$\text{min_steps} = 500.$$

Then each trajectory is sampled at 500 equally spaced indices.

So the training set becomes:

$$10 \times 500 = 5000$$

samples.

14.3 Normalization

The script normalizes stress:

$$\mathbf{S}_{\text{norm}} = \frac{\mathbf{S}}{\max |\mathbf{S}|}.$$

It also normalizes strain:

$$\mathbf{E}_{\text{norm}} = \frac{\mathbf{E}}{\max |\mathbf{E}|}.$$

This is common in neural network training because it keeps values near order one. However, there is an important mechanical caveat:

After normalization, the code computes $\mathbf{C} = 2\mathbf{E} + \mathbf{I}$ using the normalized strain. That means the principal stretches and J are computed from normalized strain, not from physical strain.

For pure regression, this can still work. But if we want the features λ_i and J to have their physical meaning, then the feature calculation should be done before normalization, or the normalization should be handled very carefully.

14.4 Energy reconstruction

The FOM database stores strain and stress, but not energy. The script therefore constructs an approximate energy by numerical integration:

$$\Delta W_i \approx \frac{1}{2} (\mathbf{S}_i + \mathbf{S}_{i-1}) : (\mathbf{E}_i - \mathbf{E}_{i-1}).$$

Then:

$$W_i = \sum_{j=1}^i \Delta W_j.$$

This is the trapezoidal rule. It is more accurate than using only the stress at the beginning or end of each increment.

14.5 Important path-dependence warning

For a perfectly hyperelastic material, the energy should be path-independent: the energy at a strain state should depend only on the final strain, not on the path used to reach it.

However, the script constructs W by integrating along each sampled trajectory. If two trajectories reach the same strain through different paths, numerical integration error, homogenization drift, or data inconsistency could produce slightly different energies.

The current training mainly uses stress loss, so this is less severe than if it trained only on W . Still, it is a useful thing to remember.

15 Training objective

15.1 The available training modes

The function `TRAIN_KAN` supports three ideas:

1. train only on stress,
2. train only on energy,
3. train on a mixed energy + stress loss.

The current settings are:

```
train_W = False
mixed_sovolev_training = False
```

Therefore, the current default is stress-based training:

$$\mathcal{L} = \frac{\text{mean} \left[(\mathbf{S}_\theta - \mathbf{S}_{\text{FOM}})^2 \right]}{\text{mean} \left[\mathbf{S}_{\text{FOM}}^2 \right] + \varepsilon}.$$

15.2 Why is this still energy-based?

Even though the loss compares stresses, the model output is still energy. The stress used in the loss is:

$$\mathbf{S}_\theta = \frac{\partial W_\theta}{\partial \mathbf{E}}.$$

So the model is trained so that the derivative of its energy matches the FOM stress.

16 Convexity

16.1 Convexity in one dimension

A scalar function $f(x)$ is convex if the line segment between any two points on the graph lies above the graph.

An intuitive example:

$$f(x) = x^2$$

is convex.

A non-convex example:

$$f(x) = \sin(x)$$

is not convex over a large interval because it curves up and down.

For a smooth one-dimensional function, convexity is equivalent to:

$$f''(x) \geq 0.$$

That means the slope $f'(x)$ is nondecreasing.

16.2 Convexity in several dimensions

For a smooth function $f(\mathbf{x})$, convexity means the Hessian matrix is positive semidefinite:

$$\nabla^2 f(\mathbf{x}) \succeq 0.$$

This means:

$$\mathbf{v}^T \nabla^2 f(\mathbf{x}) \mathbf{v} \geq 0 \quad \text{for every direction } \mathbf{v}.$$

16.3 What would convex energy imply?

If

$$\mathbf{S} = \nabla_{\mathbf{E}} W,$$

and W is convex in $\mathbf{E}$, then the stress-strain map is monotone:

$$(\mathbf{S}(\mathbf{E}_1) - \mathbf{S}(\mathbf{E}_2)) : (\mathbf{E}_1 - \mathbf{E}_2) \geq 0.$$

In simple terms: increasing strain should not produce a negative tangent energy response.

16.4 Is convexity always required?

This is subtle. In small-strain elasticity, convexity of energy in strain is a natural stability condition.

In finite-strain elasticity, requiring the energy to be convex directly in $\mathbf{F}$ is usually too strong and can be physically unrealistic. This is why polyconvexity is often discussed instead.

17 Monotonicity

17.1 One-dimensional monotonicity

A function $g(x)$ is monotone increasing if:

$$x_1 > x_2 \quad \Rightarrow \quad g(x_1) \geq g(x_2).$$

For stress-strain behavior in one dimension:

$$S = S(E),$$

monotonicity means that increasing strain does not decrease stress.

17.2 Tangent interpretation

If $S(E)$ is differentiable, monotonicity means:

$$\frac{dS}{dE} \geq 0.$$

Since $S = dW/dE$, this is:

$$\frac{d^2W}{dE^2} \geq 0.$$

So monotonicity of stress is connected to convexity of energy.

17.3 Multidimensional monotonicity

For tensor-valued stress-strain maps, monotonicity is written as:

$$(\mathbf{S}(\mathbf{E}_1) - \mathbf{S}(\mathbf{E}_2)) : (\mathbf{E}_1 - \mathbf{E}_2) \geq 0.$$

This says: when moving from strain state $\mathbf{E}_2$ to strain state $\mathbf{E}_1$, the stress change should do nonnegative work along that strain change.

18 Polyconvexity

18.1 Why polyconvexity exists

Finite-strain elasticity is usually written in terms of the deformation gradient $\mathbf{F}$. A hyperelastic energy can be written as:

$$W = W(\mathbf{F}).$$

Direct convexity in $\mathbf{F}$ is often too restrictive. It can conflict with basic physical requirements such as frame invariance.

Polyconvexity is a weaker condition that is still mathematically useful.

18.2 Beginner definition

In three dimensions, an energy $W(\mathbf{F})$ is polyconvex if it can be written as:

$$W(\mathbf{F}) = g(\mathbf{F}, \text{cof } \mathbf{F}, \det \mathbf{F}),$$

where g is convex in all of its arguments.

Here:

- $\mathbf{F}$ describes line deformation,
- $\text{cof } \mathbf{F}$ describes area deformation,
- $\det \mathbf{F}$ describes volume deformation.

The word “poly” refers to depending convexly on several deformation measures, not only on $\mathbf{F}$ itself.

18.3 Why people care about polyconvexity

Polyconvexity is important because it helps guarantee:

- existence of minimizers in nonlinear elasticity,
- better mathematical well-posedness,
- prevention of some nonphysical material instabilities,
- more robust extrapolation under large deformation.

18.4 Does the current ICKAN guarantee polyconvexity?

No clear guarantee is visible from the current script.

The code uses physically motivated inputs:

$$\bar{\lambda}_1, \quad \bar{\lambda}_2, \quad (J - 1)^2.$$

This is good, but it does not automatically prove convexity or polyconvexity.

To claim polyconvexity, the architecture or training constraints would need to be designed so that the learned energy satisfies a precise mathematical condition. The current script should be interpreted as an energy-based surrogate, not as a certified polyconvex surrogate.

19 How to read the current ICKAN code

19.1 ICKAN_surrogate.py

This file defines the model.

Step 1: import external ICKAN package.

```
sys.path.insert(0, r'C:\ICKANs')
import ickan as KAN
```

This path is Windows-specific. On Linux, this import fails unless the package `ickan` is installed or the path is changed.

Step 2: build KAN for energy.

```
self.KAN_W = KAN.MultKAN(...)
```

This is the neural network that outputs W .

Step 3: convert strain to KAN inputs.

```

E[:, 0, 0] = strain[:, 0]
E[:, 1, 1] = strain[:, 1]
E[:, 0, 1] = 0.5 * strain[:, 2]
E[:, 1, 0] = 0.5 * strain[:, 2]

C = 2.0 * E + torch.eye(2)
J = torch.linalg.det(C)**0.5
square_eigenvalues = torch.linalg.eigvalsh(C)
eigenvalues = torch.sqrt(square_eigenvalues)

```

This reconstructs the mechanics features.

Step 4: evaluate energy.

```

W_raw = self.KAN_W.forward(kan_input)
return W_raw - W0

```

The subtraction makes the energy zero at the reference state.

Step 5: get stress by autograd.

```

predicted_stress = torch.autograd.grad(
    outputs=W,
    inputs=strain_database,
    grad_outputs=torch.ones_like(W),
    create_graph=True
)[0]

```

This is the central hyperelastic idea.

19.2 run_ICKAN_surrogate.py

This file does the experiment.

Step 1: load data. It loops over trajectories 1 to 10 and loads:

$$\mathbf{E}_i(t), \quad \mathbf{S}_i(t).$$

Step 2: downsample. Every trajectory is reduced to 500 points.

Step 3: normalize. It scales strain and stress to order-one values.

Step 4: integrate energy. It computes approximate W from stress-strain work.

Step 5: train ICKAN. It trains the energy model, using stress error as the default loss.

Step 6: save plots. It saves:

- predicted energy versus reference integrated energy,
- predicted stress versus FOM stress,
- spline/KAN plots,
- model weights.

20 Important caveats in the current scripts

20.1 Caveat 1: external dependency path

The file `ICKAN_surrogate.py` contains:

```
C:\ICKANs.
```

This is not portable. The code will only run where that path exists or where `ickan` is installed.

20.2 Caveat 2: output directory

The script saves files into:

```
ICKAN_predictions/.
```

But the script does not explicitly create that folder. It should call something like:

```
os.makedirs("ICKAN_predictions/splines", exist_ok=True)
```

20.3 Caveat 3: computed strain versus applied strain

The script uses:

```
trajectory_i_strain.npy.
```

This is the homogenized computed strain. The imposed boundary strain is:

```
trajectory_i_applied_strain.npy.
```

Which one should be used depends on the intended final use.

- If the surrogate will be queried by prescribed macro strain, use applied strain.
- If the surrogate is intended to map homogenized measured strain to homogenized stress, use computed strain.

For most constitutive surrogate use cases, the cleaner input is probably the applied macroscopic strain.

20.4 Caveat 4: normalization and physical features

The model computes $\mathbf{C}$, J , and stretches from normalized strain. This makes the inputs numerically convenient, but the quantities are no longer the physical $\mathbf{C}$, J , and stretches.

A more physically transparent workflow would be:

1. start from physical strain,
2. compute physical invariants/stretching,
3. normalize those features for neural network training,
4. save the normalization constants.

20.5 Caveat 5: no train/test split

The current script trains and plots using the same data. This checks whether the network can fit the database, but it does not test generalization.

A better test would be:

- train on 8 trajectories,
- validate on 1 trajectory,
- test on 1 trajectory,
- or leave out an entire G_{xy} level.

20.6 Caveat 6: not certified convex or polyconvex

The model is energy-based, but that alone does not guarantee:

- convexity,
- monotonicity,
- polyconvexity,
- positive tangent stiffness everywhere.

Those properties require either special architecture constraints, special loss terms, or post-training verification.

21 What would a good ICKAN test report include?

A useful report should include:

1. which trajectories were used for training,
2. which trajectories were held out for validation/testing,
3. whether inputs are applied strain or homogenized computed strain,
4. strain and stress normalization constants,
5. whether invariants are computed before or after normalization,
6. stress relative error per component,
7. energy error if energy is used,
8. monotonicity or tangent checks along selected paths,
9. plots for unseen trajectories,
10. saved model weights and reproducibility seed.

22 Recommended next steps

22.1 Step 1: make the code portable

Replace the hard-coded Windows path with a documented installation or a local relative path.
For example:

```
try:
    import ickan as KAN
except ImportError as exc:
    raise ImportError(
        "Could not import 'ickan'. Install it or add its path to PYTHONPATH."
    ) from exc
```

22.2 Step 2: create output directories

Before saving:

```
os.makedirs("ICKAN_predictions/splines", exist_ok=True)
```

22.3 Step 3: decide the input strain

For a constitutive law, prefer:

`trajectory_i_applied_strain.npy`

unless there is a specific reason to use homogenized computed strain.

22.4 Step 4: save scalars

If strain and stress are normalized, save:

- strain scale,
- stress scale,
- energy scale,
- feature scales.

Without these, it is hard to use the model later in a physical simulation.

22.5 Step 5: add a real test

For example:

- train on trajectories 1–8,
- validate on trajectory 9,
- test on trajectory 10.

Or more physically:

- train on $G_{xy} = 0, 0.05, -0.05$,
- test on $G_{xy} = 0.1, -0.1$.

23 Mental model for Sebastian

The easiest way to think about the entire pipeline is:

1. The FOM solver is an expensive laboratory.
2. The laboratory deforms the perforated RVE in many ways.
3. For each deformation, it reports average strain and average stress.
4. The ICKAN tries to learn the hidden effective material energy of the perforated RVE.
5. Once the energy is learned, stress is obtained by differentiation.

So the ICKAN is not replacing the mesh directly. It is replacing the final homogenized constitutive relationship:

$$\mathbf{E} \mapsto \mathbf{S}.$$

24 Glossary

RVE Representative volume element. A small sample used to infer macroscopic material behavior.

Homogenization Averaging a detailed local field to obtain effective macroscopic quantities.

Green–Lagrange strain A finite-strain measure $\mathbf{E} = \frac{1}{2}(\mathbf{C} - \mathbf{I})$.

Second Piola–Kirchhoff stress A stress measure work-conjugate to Green–Lagrange strain.

Work-conjugate Two quantities are work-conjugate if their product gives mechanical work or energy increment. Here $\mathbf{S} : d\mathbf{E}$.

Deformation gradient Tensor $\mathbf{F}$ that maps reference differential vectors to current differential vectors.

Right Cauchy–Green tensor Tensor $\mathbf{C} = \mathbf{F}^T \mathbf{F}$ measuring stretch without rigid-body rotation.

Principal stretches Stretch factors λ_i in the principal directions of deformation.

Invariant A scalar quantity that does not change under coordinate rotation.

Hyperelasticity A material theory where stress is obtained as the derivative of an energy.

Convexity A mathematical property meaning the function has no downward curvature. For smooth scalar functions, this is related to nonnegative second derivative or positive semidefinite Hessian.

Monotonicity A property meaning the output does not decrease when the input increases, generalized to tensors through nonnegative work of increments.

Polyconvexity A finite-strain stability condition where the energy is convex in $\mathbf{F}$, $\text{cof } \mathbf{F}$, and $\det \mathbf{F}$ through another function.

KAN Kolmogorov–Arnold Network. A network using learnable scalar functions, often splines, on edges.

ICKAN The specific ICKAN library/model used by Alejandro’s script. In this repository, it is used as an energy-based constitutive surrogate.

25 Final interpretation

Alejandro's current ICKAN test is best interpreted as a first prototype for:

learning the effective hyperelastic energy of the perforated RVE

from your FOM-generated homogenized strain-stress data.

The idea is scientifically reasonable because the FOM material is hyperelastic and the RVE response should have an effective energy-like behavior under the controlled loading paths. However, the current implementation should not yet be interpreted as a validated, portable, poly-convex constitutive model. It is an energy-based regression experiment that needs careful validation.